TA212

Project Report

Robotic Arm

(Prof. Mohit Law)

Group G5 (Tuesday)

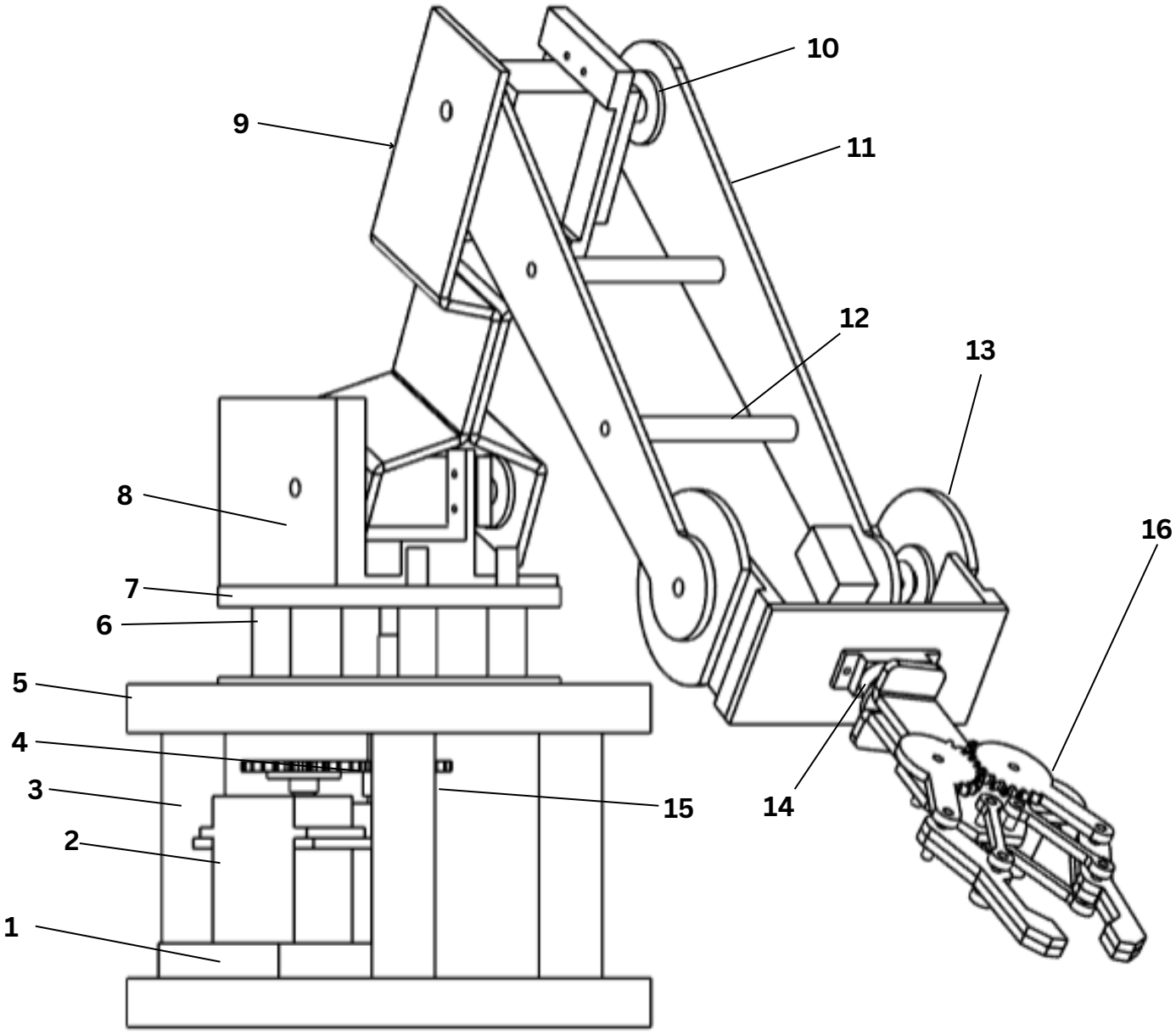
- Bala Rohit (240255)
- Krishna Sharma (240564)
- Lakshya Arukiya (240589)
- Pradeep Bishnoi (240756)
- Harshwardhan Giri Goswami (240444)
- Ujjwal Raj (241112)
- Sunand Garg (241061)

Project Overview

The primary objective of this project is to develop an advanced 6-degree-of-freedom robotic arm that utilizes a computer vision interface for intuitive, contactless gesture-based control. The mechanical assembly consists of 21 distinct manufactured parts, featuring a robust foundation of ACP base plates and a high-density steel shaft to ensure structural stability during heavy base movements. While the out-of-pocket expenditure for newly purchased electrical components was strictly limited to ₹2,250, the actual comprehensive value of the prototype, factoring in all raw materials and extensive machining time for 3D printing and turning is calculated at ₹6,155. Potential future improvements for this system could focus on integrating high-resolution encoders for closed-loop positional feedback to eliminate servo jitter and implementing a wireless communication protocol to allow for remote operation in hazardous environments.

Table of Contents

Isometric Full view



Part no.	Part Name	Material	Qty
1	Mounting 1	3D Printed	1
2	Servo (MG996R)	Purchased	3
3	Base Support	3D Printed	3
4	Gear (a,b)	3D Printed	2
5	Base plate A	ACP sheet	2
6	Spacer plate	3D Printed	3
7	Base plate B	3D Printed	2
8	Mounting 2	3D Printed	1
9	Arm (a,b)	3D Printed	2
10	Motor Head	Purchased	6
11	Front Arm (a,b)	3D Printed	2
12	Upper Arm Spacer	3D Printed	2
13	Wrist Assemble	3D Printed	1
14	Servo Motor (SG90)	Purchased	3

15	shaft	Metal	1
16	Gripper	3D Printed	1

Detailed Torque Calculation and rationale for usage of electronics

1. Mass Estimation

Component	Mass (kg)
Upper Arm (PLA)	0.04
Forearm (PLA)	0.04
Wrist + Gripper	0.05
MG996R Motors (3)	0.165
SG90 Motors (3)	0.027

2. Torque Formula

Torque is calculated using: $\tau = \Sigma(m \times g \times r)$, where $g = 9.81\ m/s^2$.

3.1 Gripper Joint

$\tau = W \times 9.81 \times 0.05$

Setting $\tau = 0.25\ N \cdot m \Rightarrow W = 0.25 / (9.81 \times 0.05) \approx 0.51\ kg$

3.2 Wrist Joint

$\tau = (0.059 + W) \times 9.81 \times 0.1$

$\Rightarrow (0.059 + W) \times 0.981 = 0.25$

$\Rightarrow W \approx 0.196\ kg$

3.3 Elbow Joint

$\tau = (0.04 \times 9.81 \times 0.09) + (0.05 \times 9.81 \times 0.18) + (0.009 \times 9.81 \times 0.18) + (W \times 9.81 \times 0.2)$

$\tau = 0.139 + 1.962W$

$\Rightarrow 0.139 + 1.962W = 1.08$

$\Rightarrow W \approx 0.48\ kg$

3.4 Shoulder Joint (Critical)

$\tau = (0.04 \times 9.81 \times 0.075) + (0.04 \times 9.81 \times 0.20) + (0.05 \times 9.81 \times 0.30) + (0.11 \times 9.81 \times 0.25) + (0.027 \times 9.81 \times 0.30) + (W \times 9.81 \times 0.33)$

$\tau = 0.603 + 3.237W$

$\Rightarrow 0.603 + 3.237W = 1.08$

$\Rightarrow W \approx 0.147$ kg

4. Final Result

Maximum payload is limited by shoulder joint ≈ 120–150 grams.

5. Electronics Justification

Arduino Uno is used for control and serial communication.

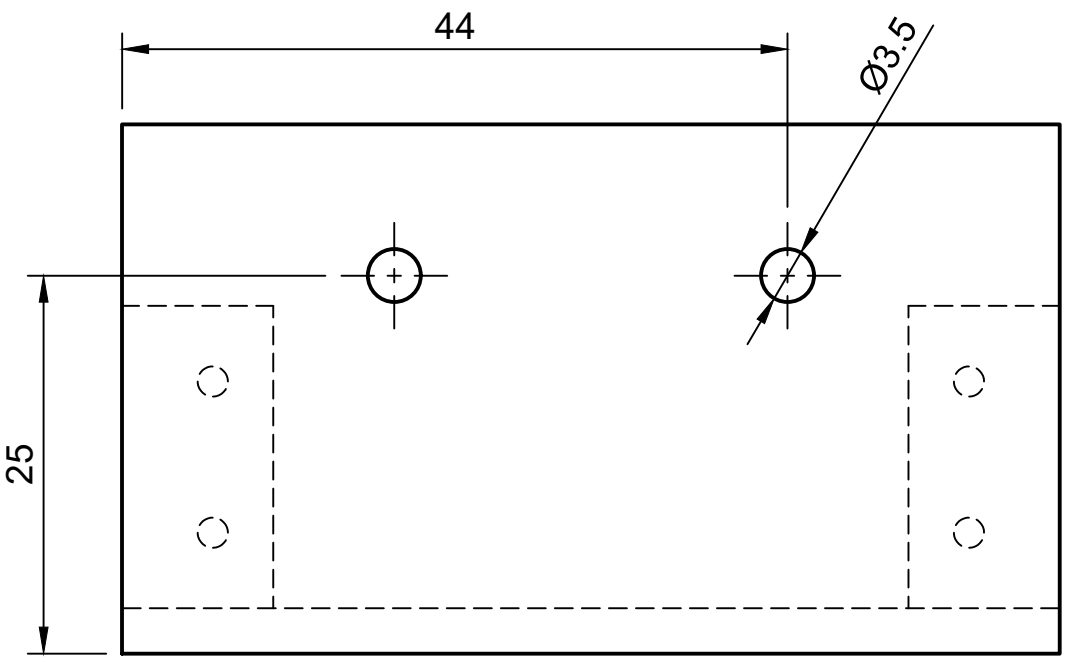
PCA9685 driver ensures stable multi-servo PWM control at 50 Hz.

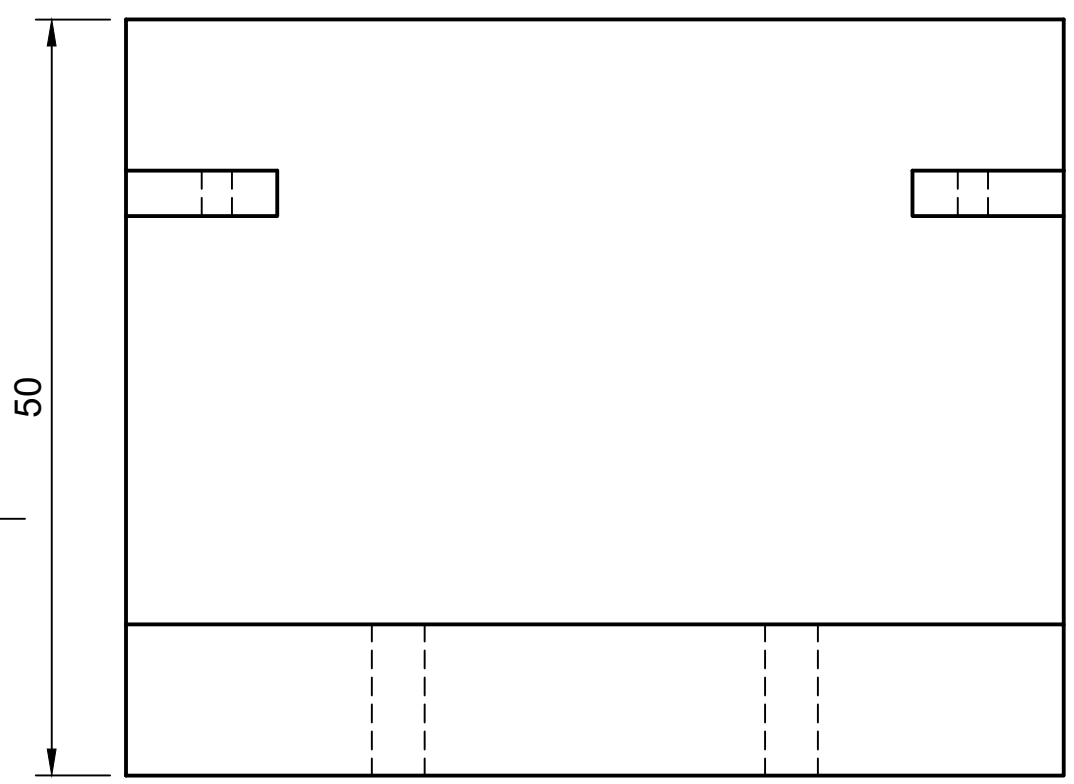
Python + OpenCV enables gesture recognition.

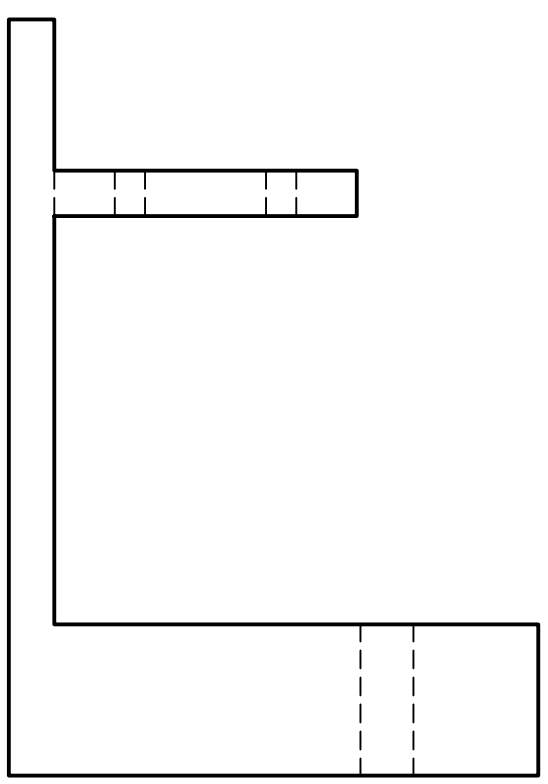
Li-ion batteries provide sufficient current for servo motors.

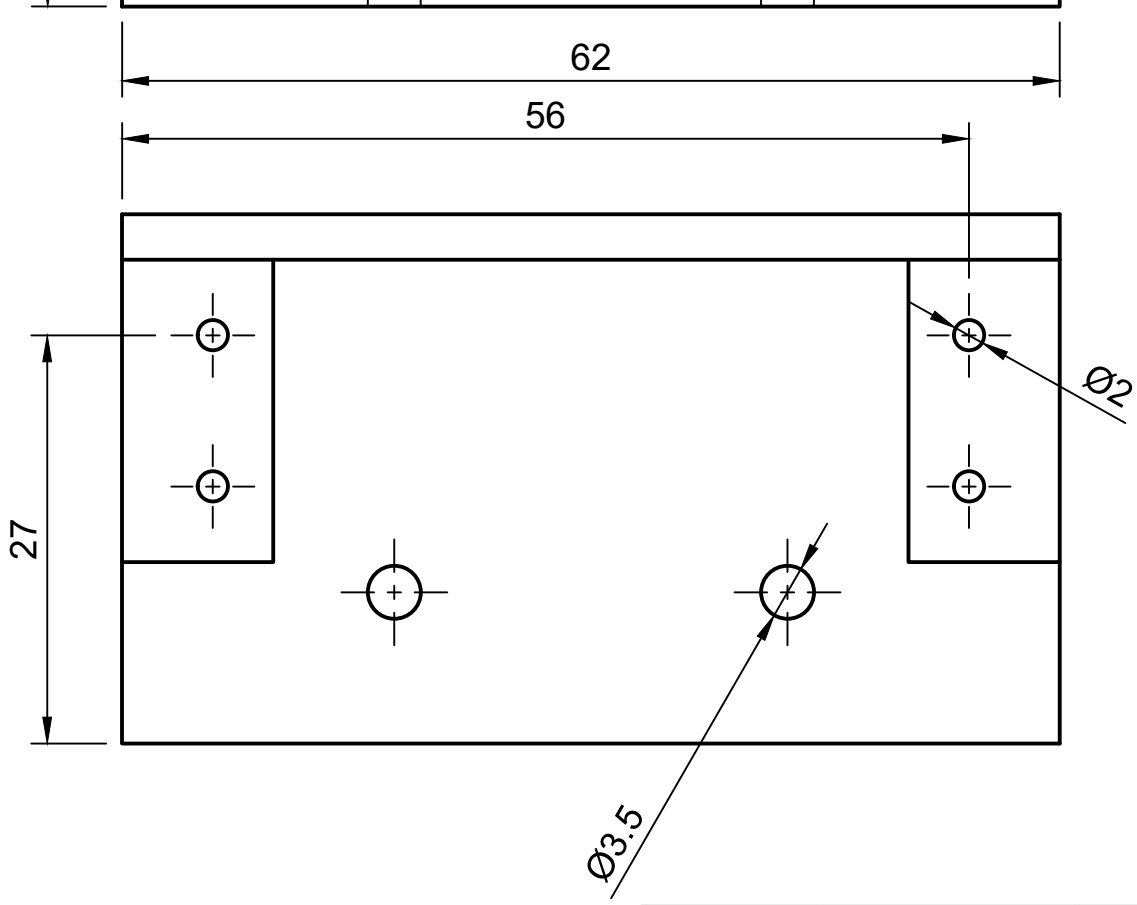
7. Conclusion

The system is torque-limited at the shoulder joint. The design is optimized for lightweight operation and gesture-based control.

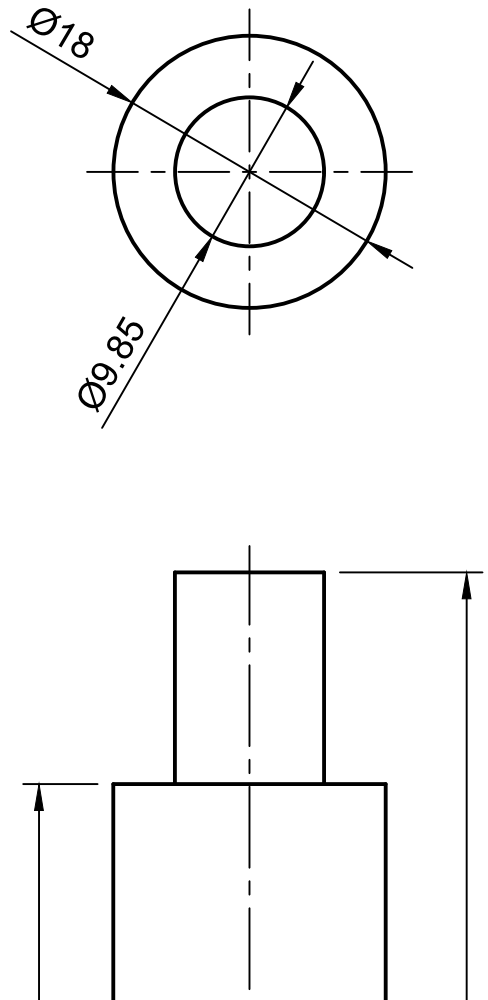




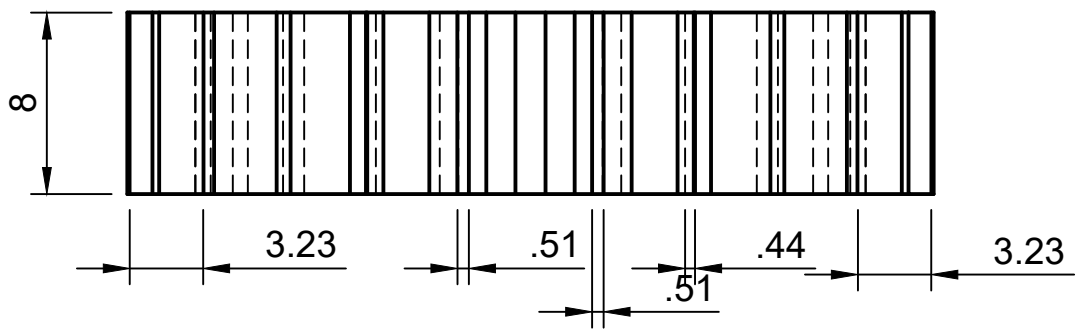


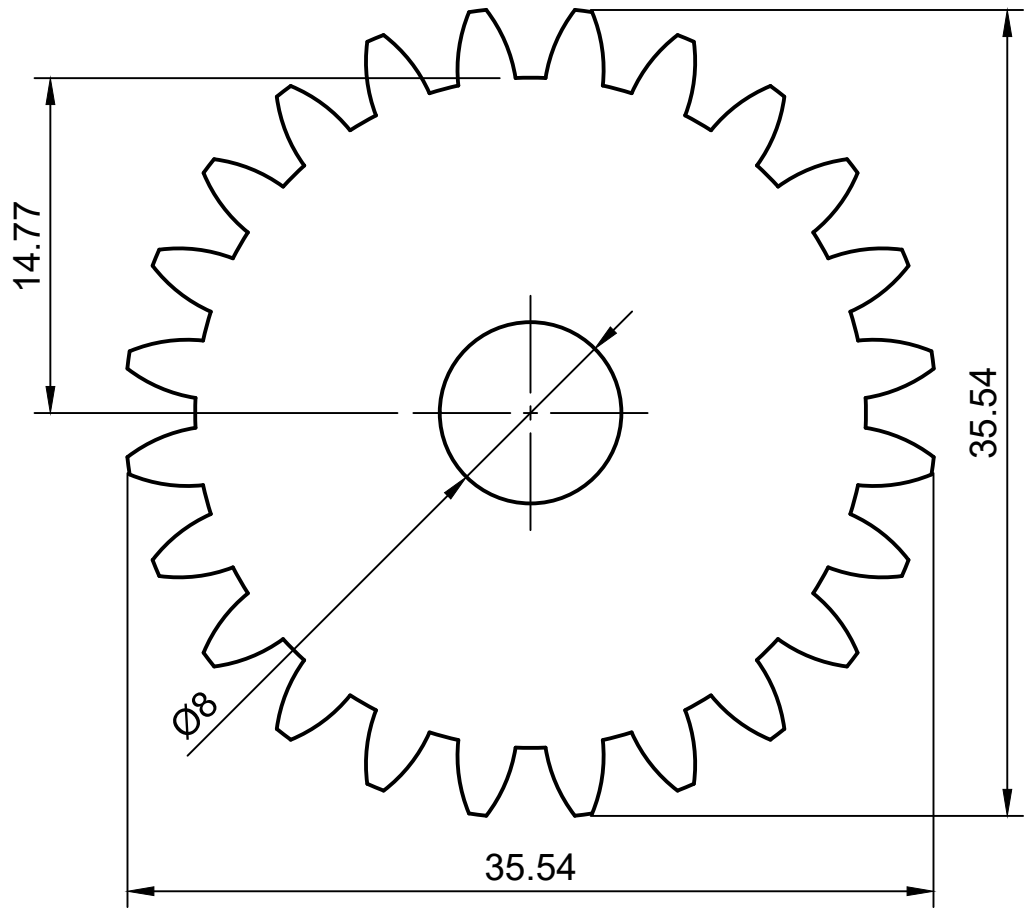


Dept.	Technical reference	Created by	Approved by
		G5 20-04-2026	
		Document type	Document status
		Title: Mounting 1	DWG No.
			Rev. / Date of issue / Sheet: 1/1

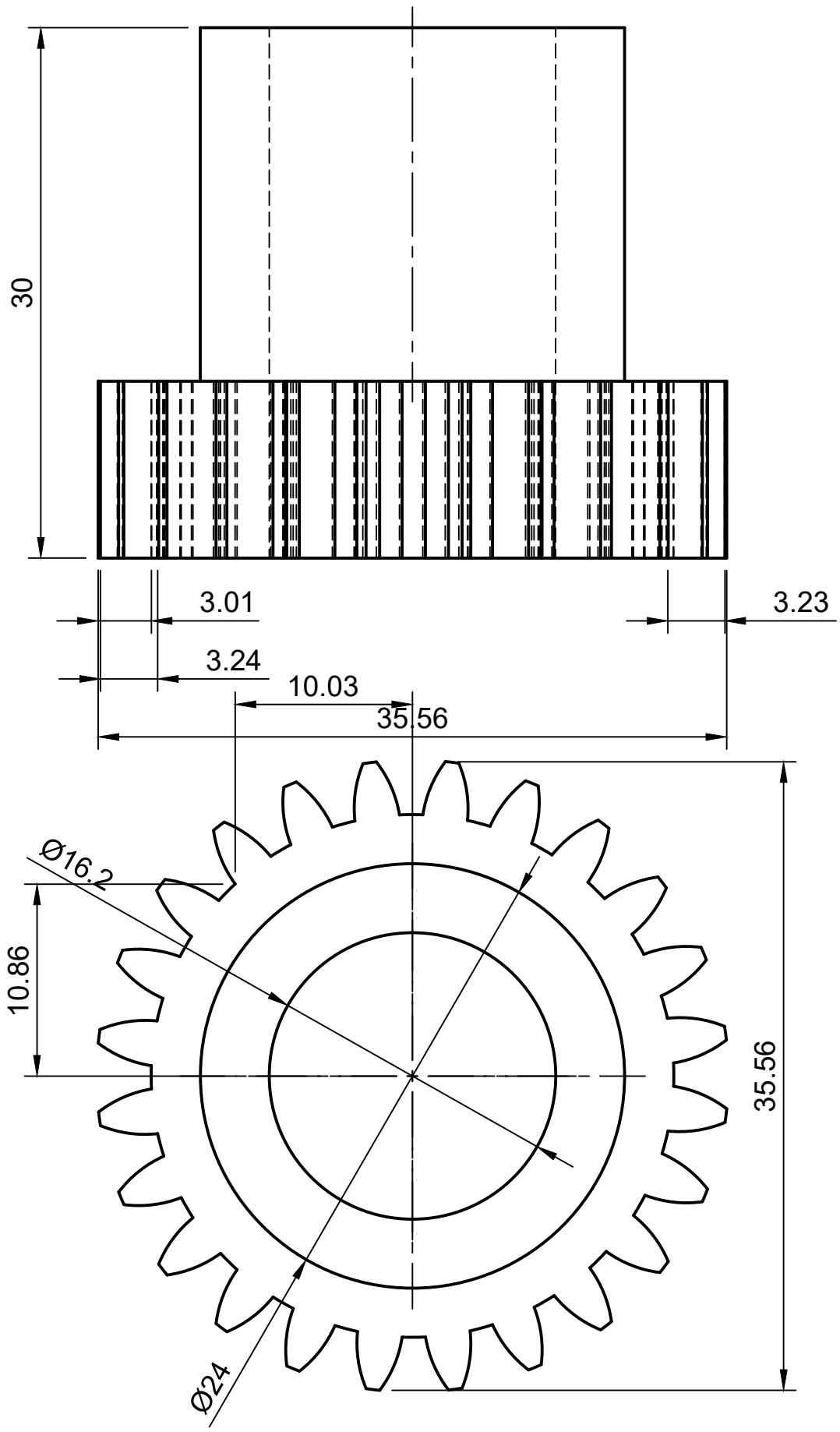


Dept.	Technical reference	Created by	Approved by
		G5 20-04-2026	
		Document type	Document status
		Title: Base Support	DWG No.: 3
			Rev. / Date of issue / Sheet: 1/1

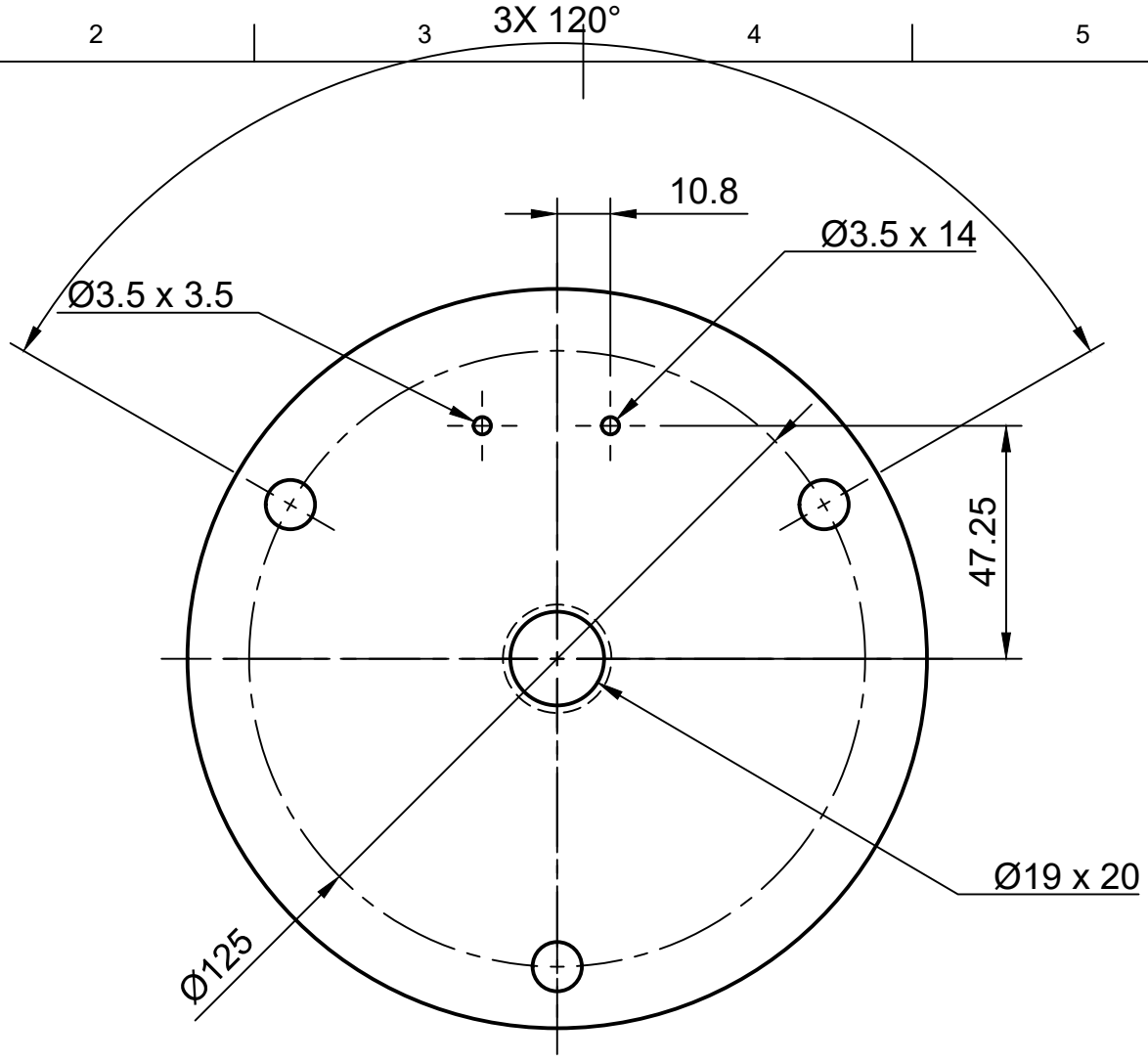


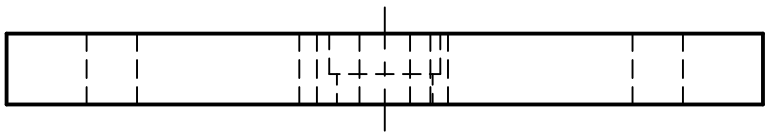


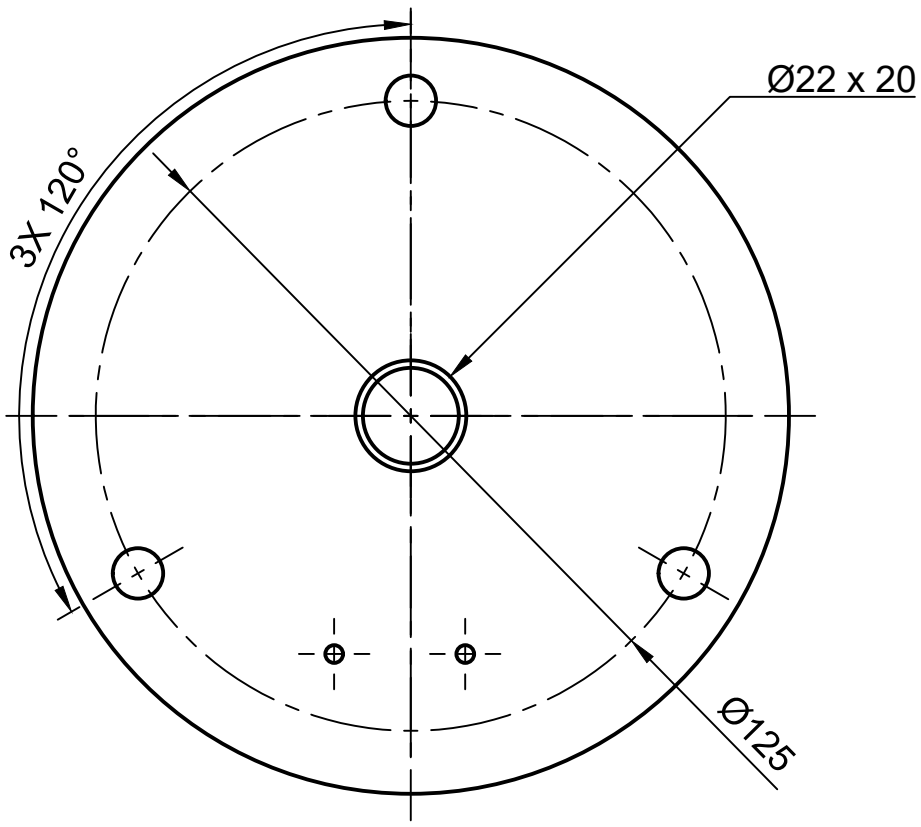
Dept.	Technical reference	Created by	Approved by
		G5 20-04-2026	
		Document type	Document status
		Title: Base Gear	DWG No.
			Rev. / Date of issue / Sheet: 1/1



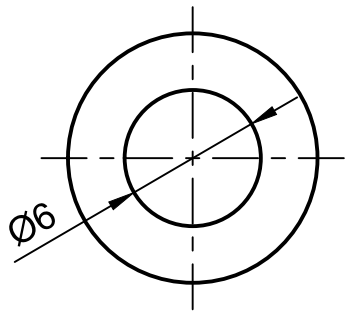
Dept.	Technical reference	Created by	Approved by
		G5 20-04-2026	
		Document type	Document status
		Title: Shaft Gear	DWG No.
			Rev. / Date of issue / Sheet: 1/1

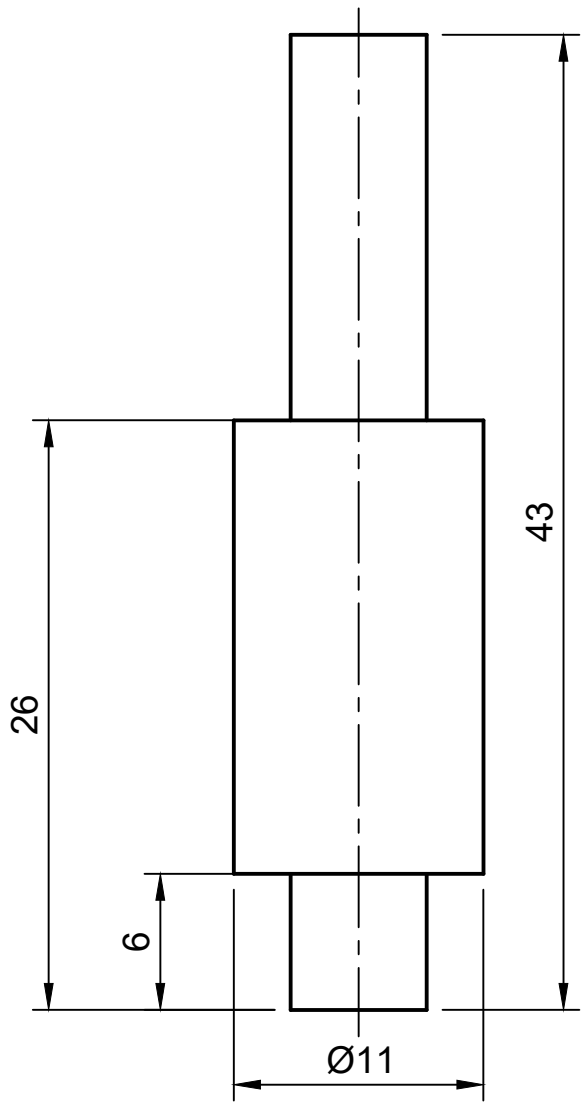




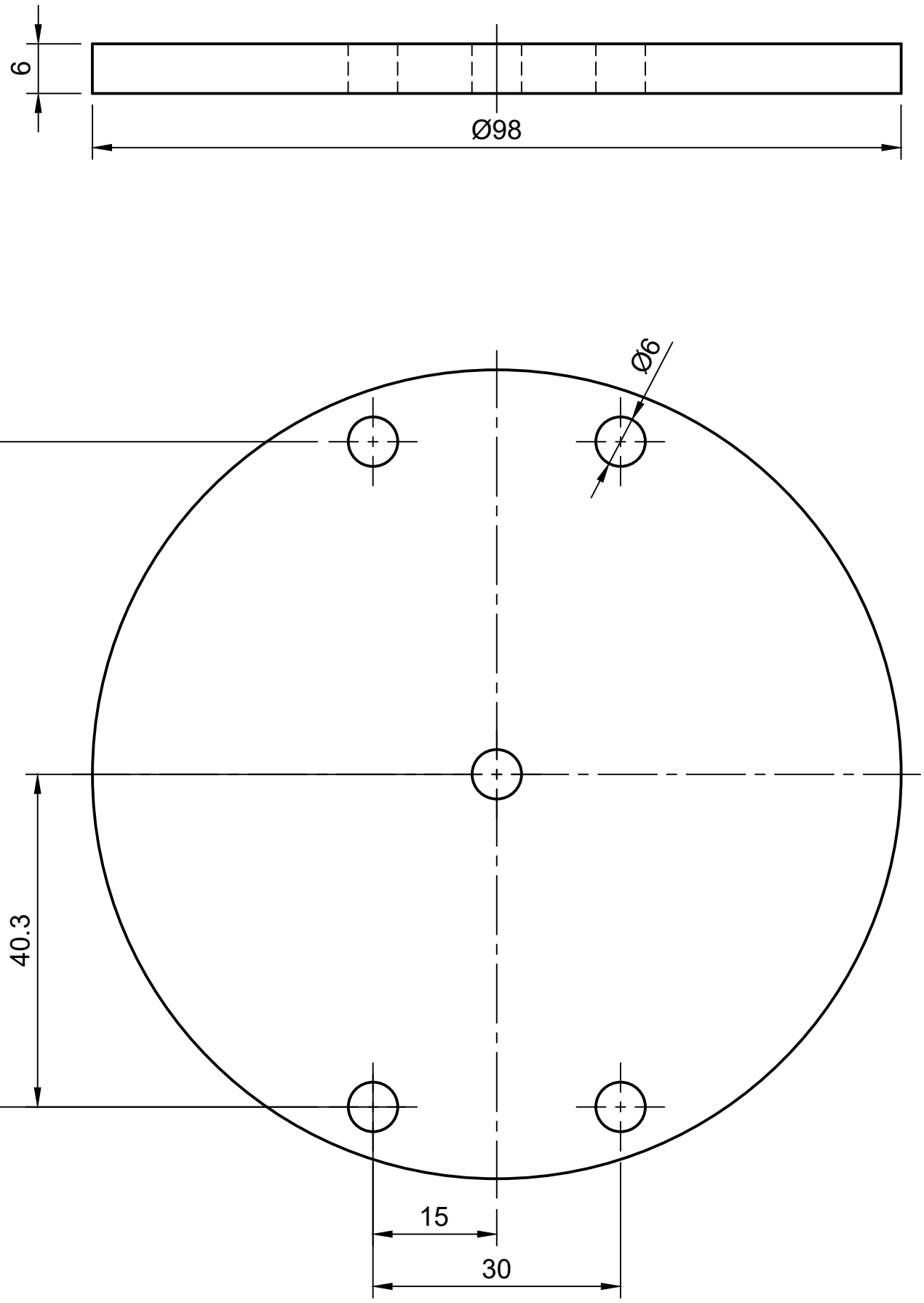


Dept.	Technical reference	Created by	Approved by
		G5 20-04-2026	
		Document type	Document status
		Title: Base Plate A	DWG No.: 5
			Rev. / Date of issue / Sheet: 1/1

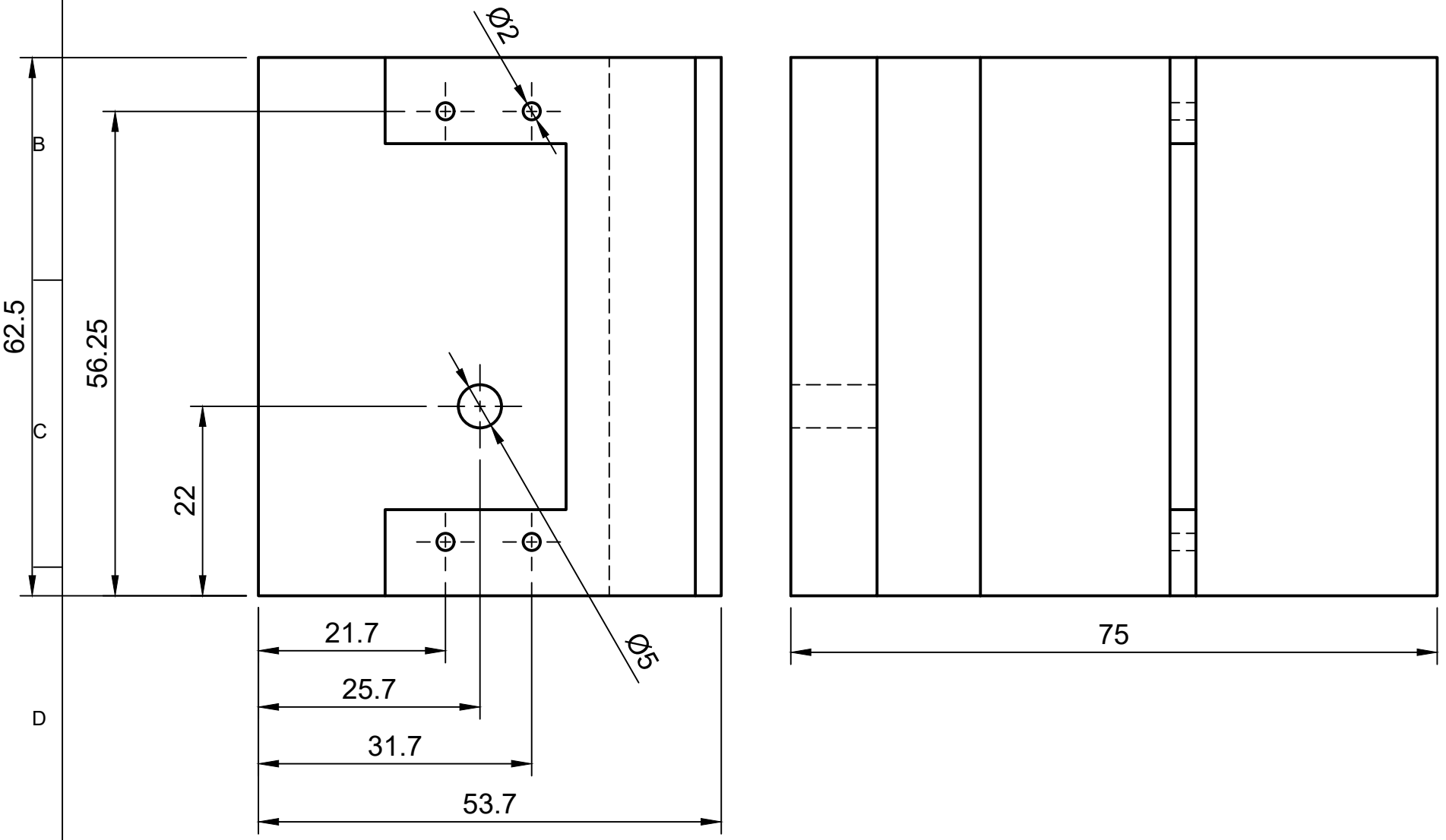


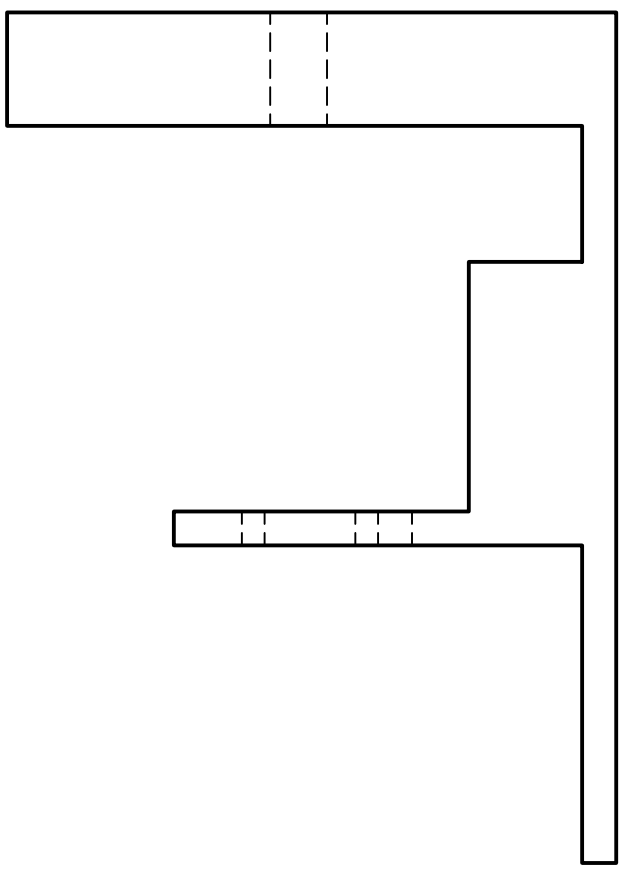


Dept.	Technical reference	Created by	Approved by
		G5 20-04-2026	
		Document type	Document status
		Title: Spacer Plate	DWG No.: 6
			Rev. / Date of issue / Sheet: 1/1

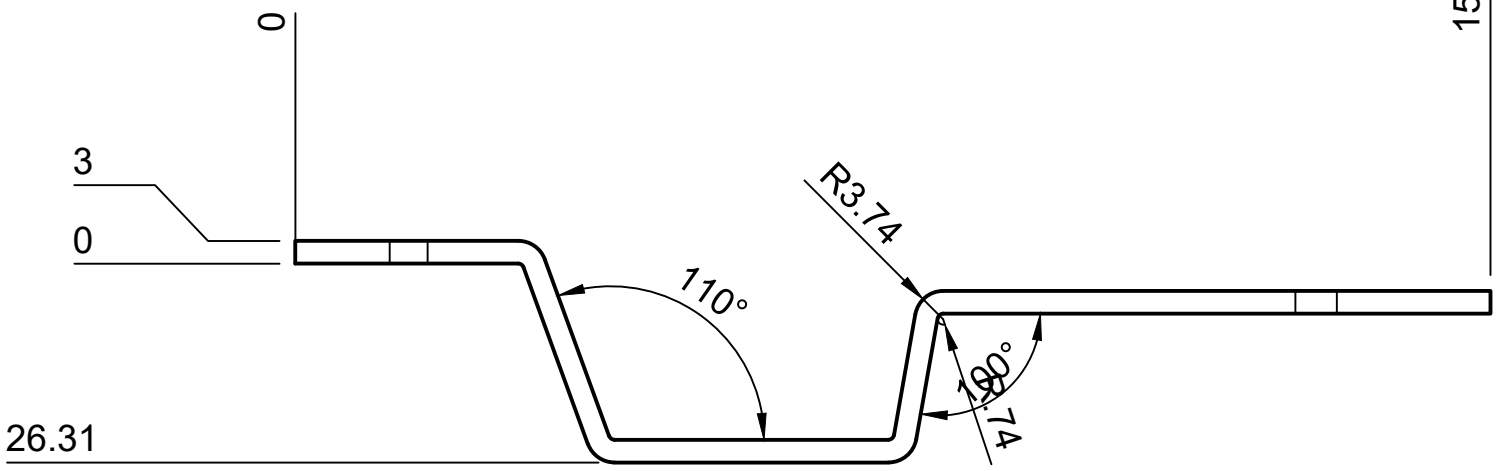


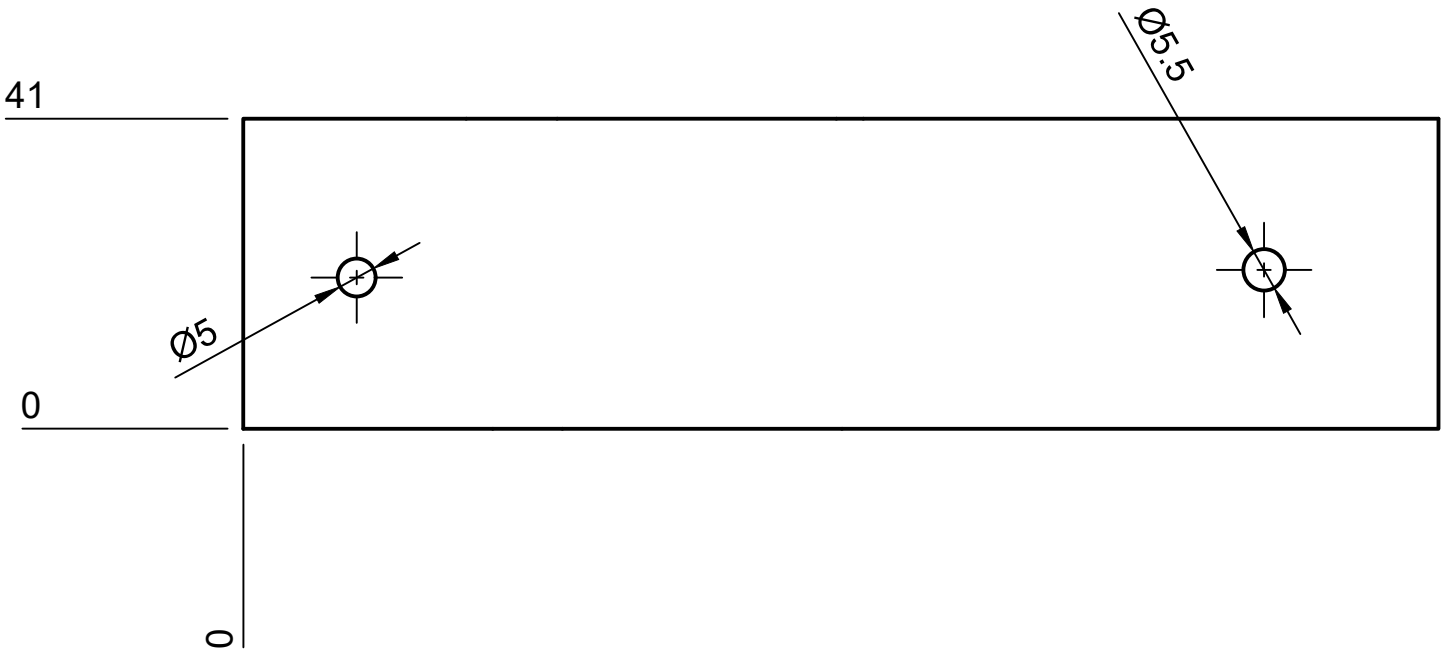
Dept.	Technical reference	Created by	Approved by
		G5 20-04-2026	
		Document type	Document status
		Title: Base Plate B	DWG No.: 7
			Rev. / Date of issue / Sheet: 1/1



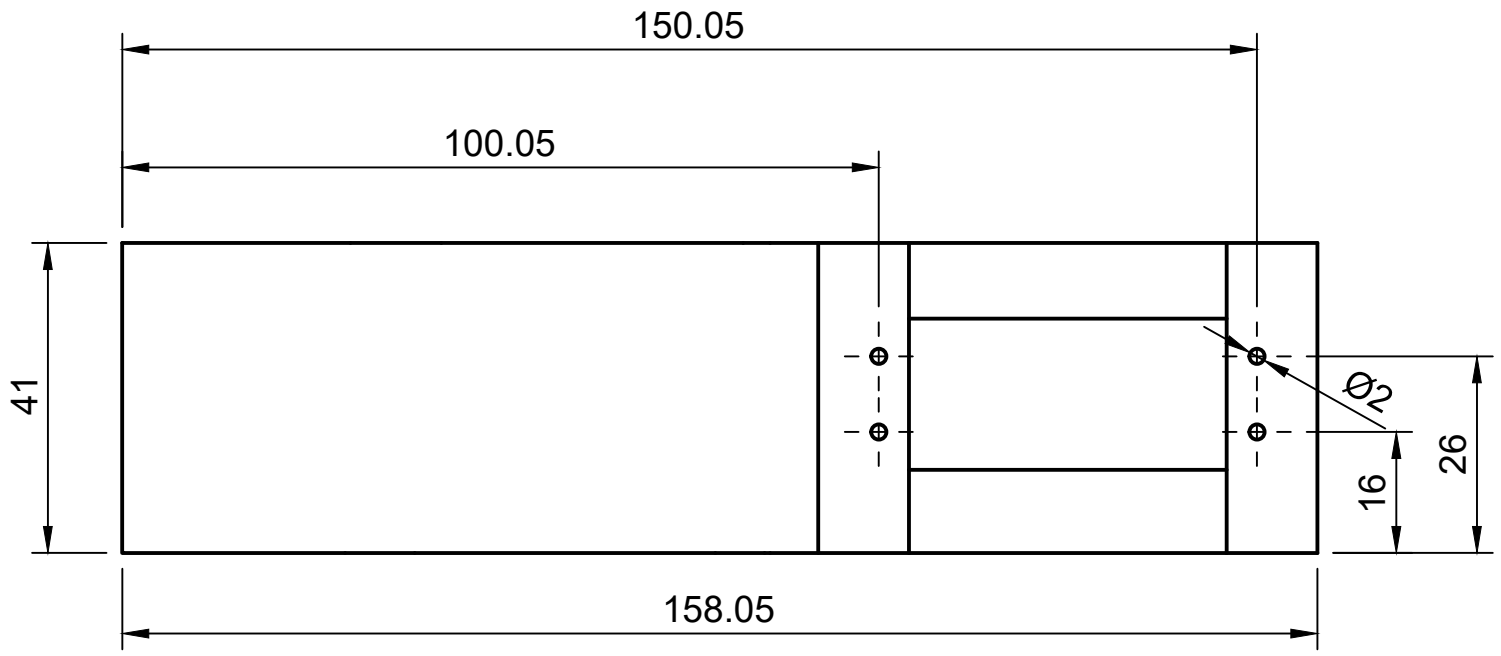


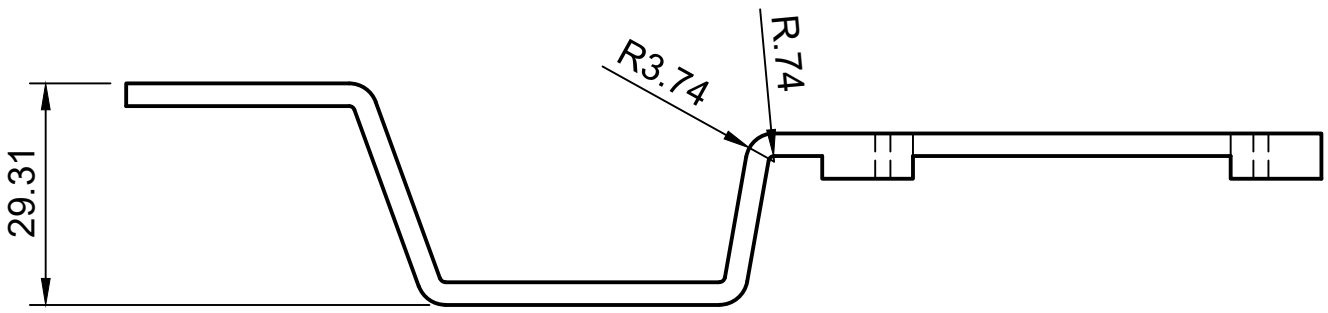
Dept.	Technical reference	Created by	Approved by
		G5 20-04-2026	
		Document type	Document status
		Title: Mounting 2	DWG No.: 8
			Rev. / Date of issue / Sheet: 1/1



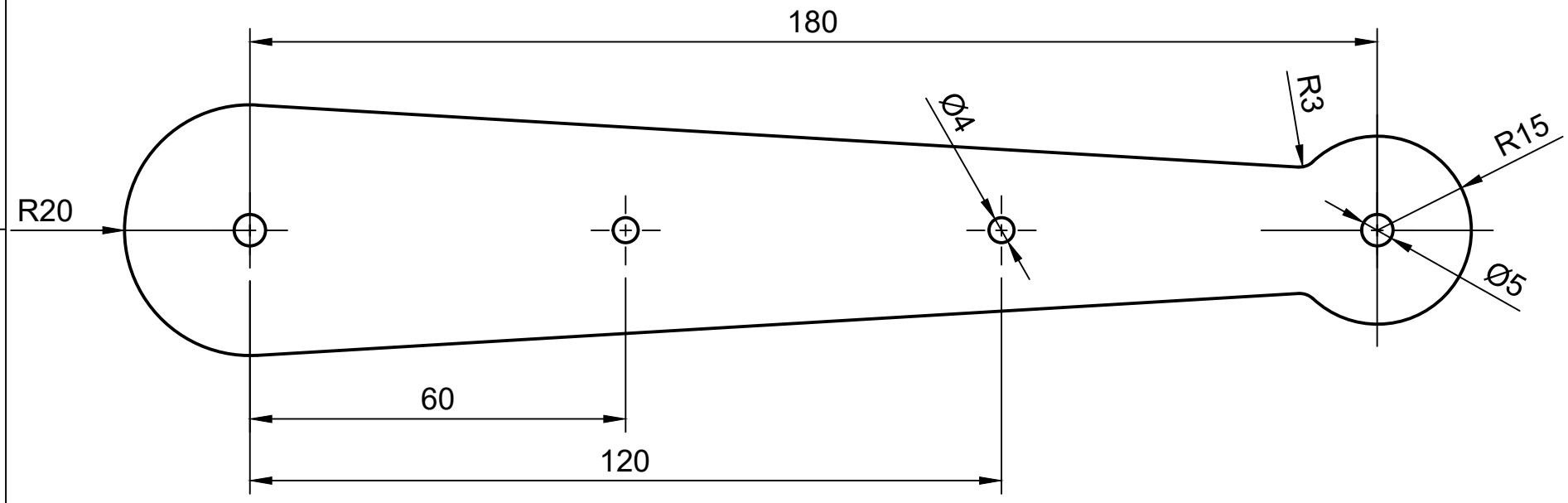


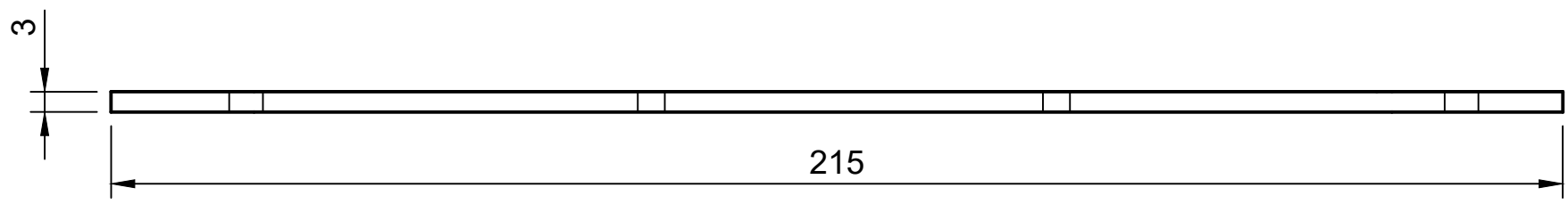
Dept.	Technical reference	Created by	Approved by
		G5 20-04-2026	
		Document type	Document status
		Title: Arm	DWG No.: 9 a
			Rev. / Date of issue / Sheet: 1/1



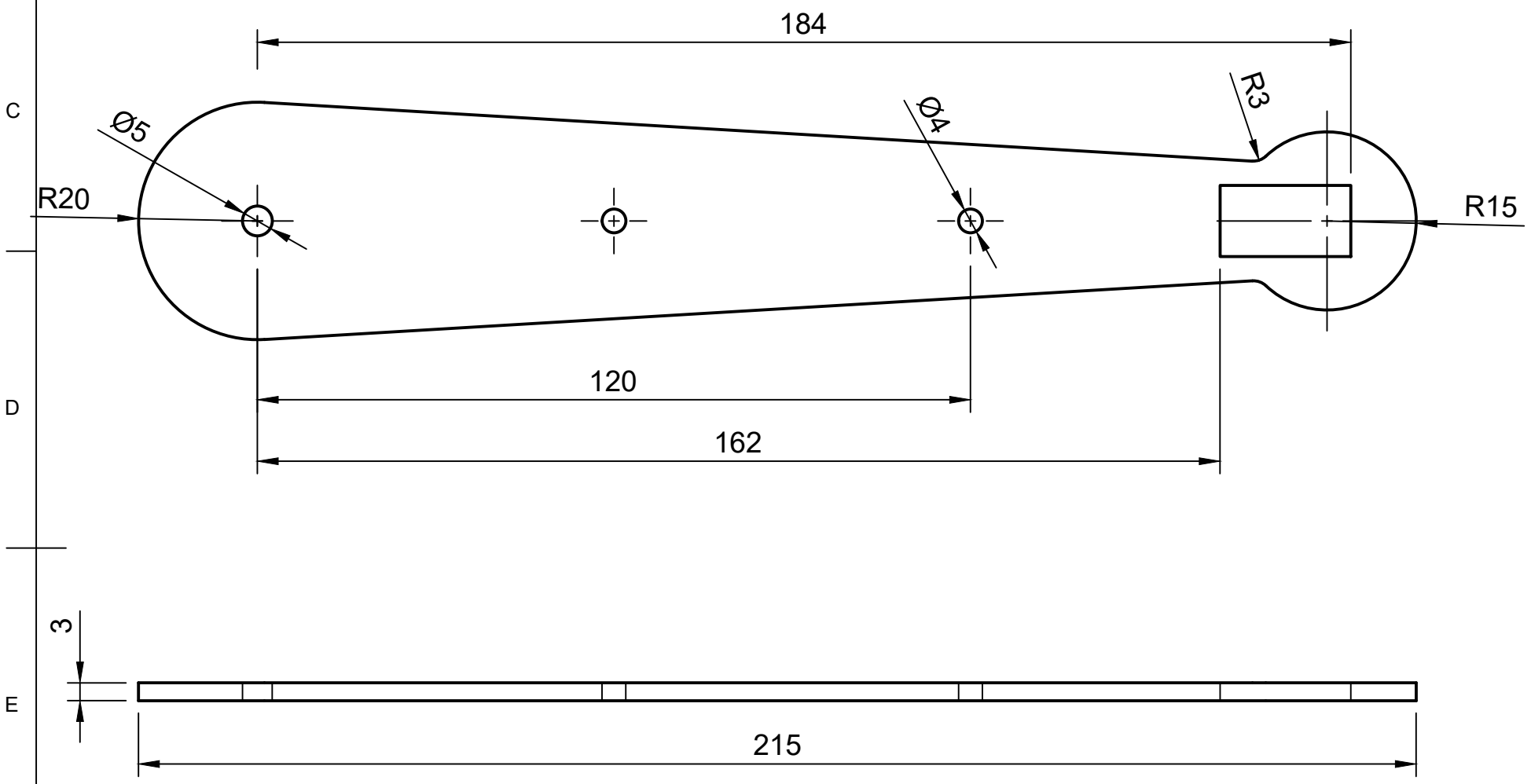


Dept.	Technical reference	Created by	Approved by
		G5 20-04-2026	
		Document type	Document status
		Title: Arm	DWG No.: 9 b
			Rev. / Date of issue / Sheet: 1/1

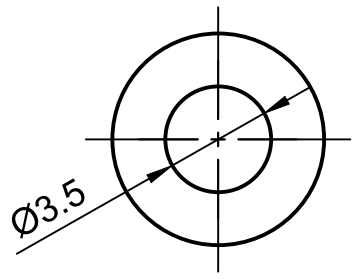


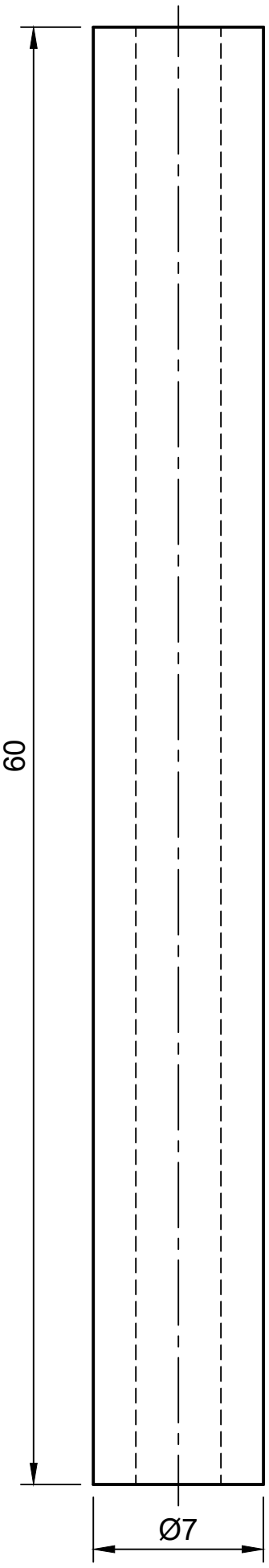


Dept.	Technical reference	Created by	Approved by
		G5 20-04-2026	
		Document type	Document status
		Title: Front Arm	DWG No.: 11 a
		Rev.	Date of issue / Sheet: 1/1

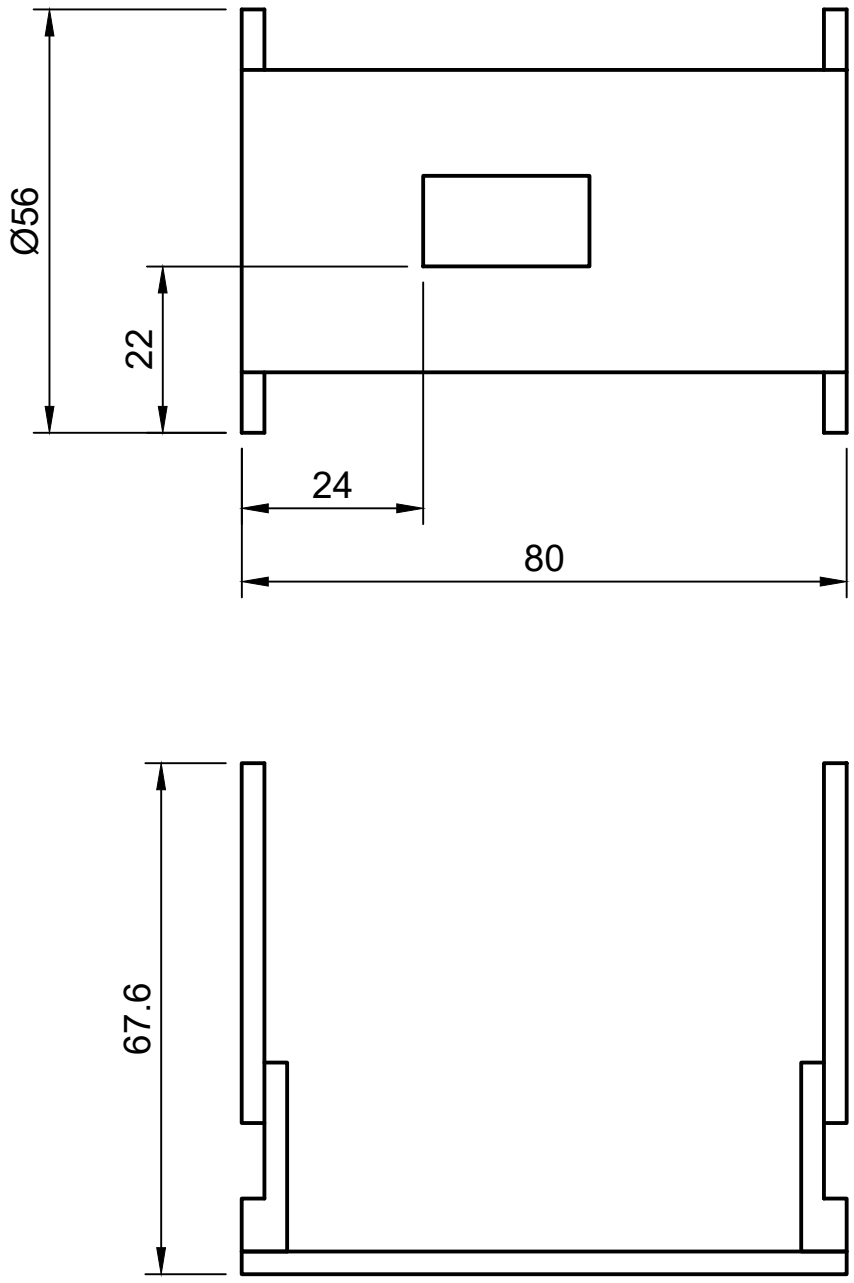


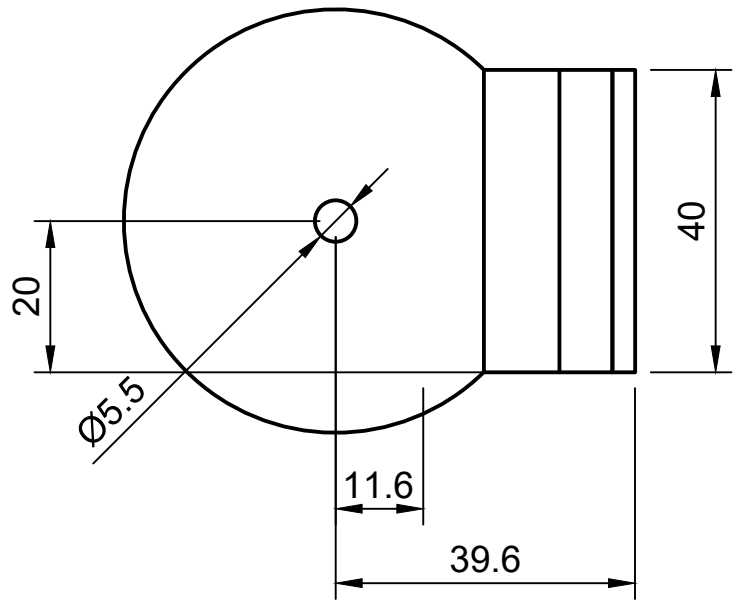
Dept.	Technical reference	Created by	Approved by
		G5 20-04-2026	
		Document type	Document status
		Title: Front Arm	DWG No.: 11 b
		Rev.	Date of issue / Sheet: 1/1



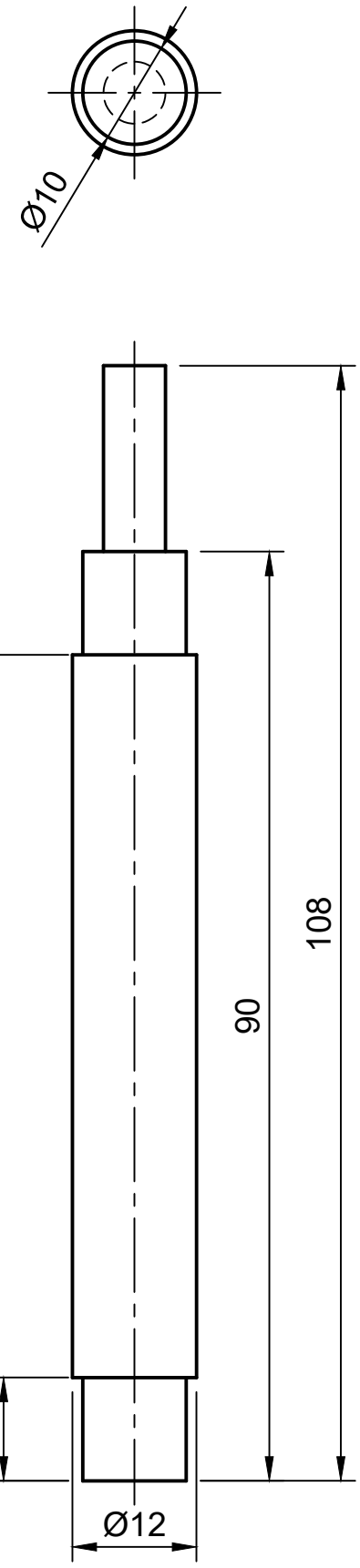


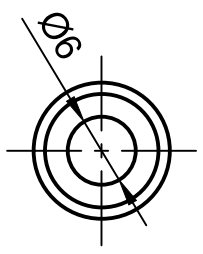
Dept.	Technical reference	Created by G5 20-04-2026	Approved by
		Document type	Document status
		Title: Upper Arm Spacer	DWG No.: 12
			Rev. / Date of issue / Sheet: 1/1



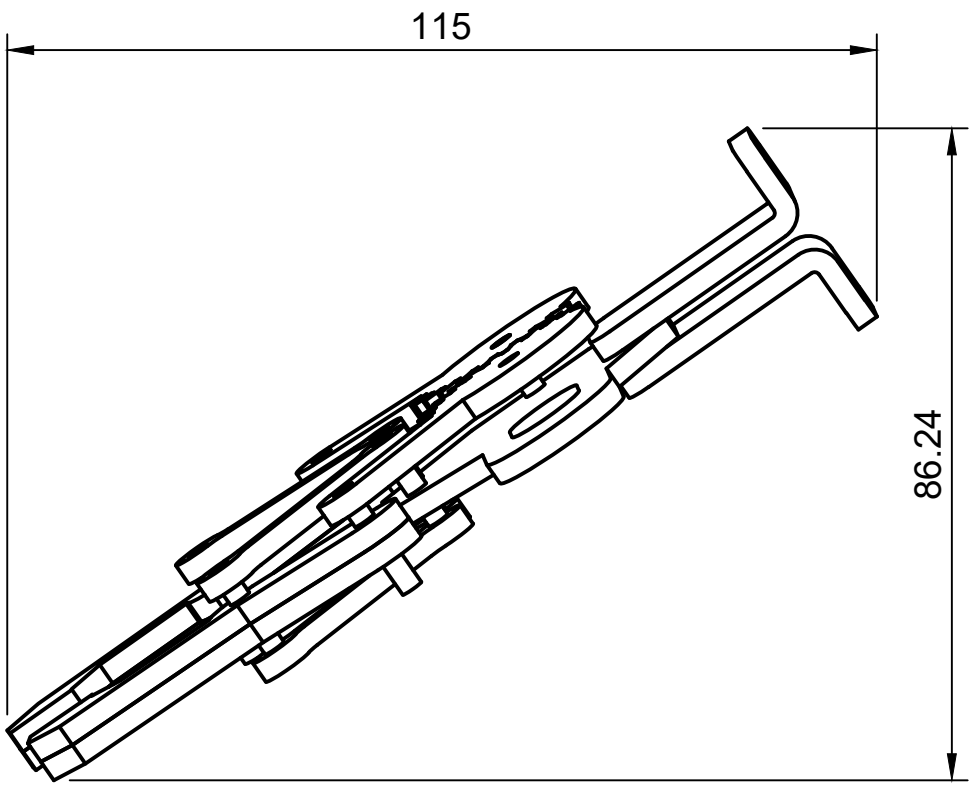


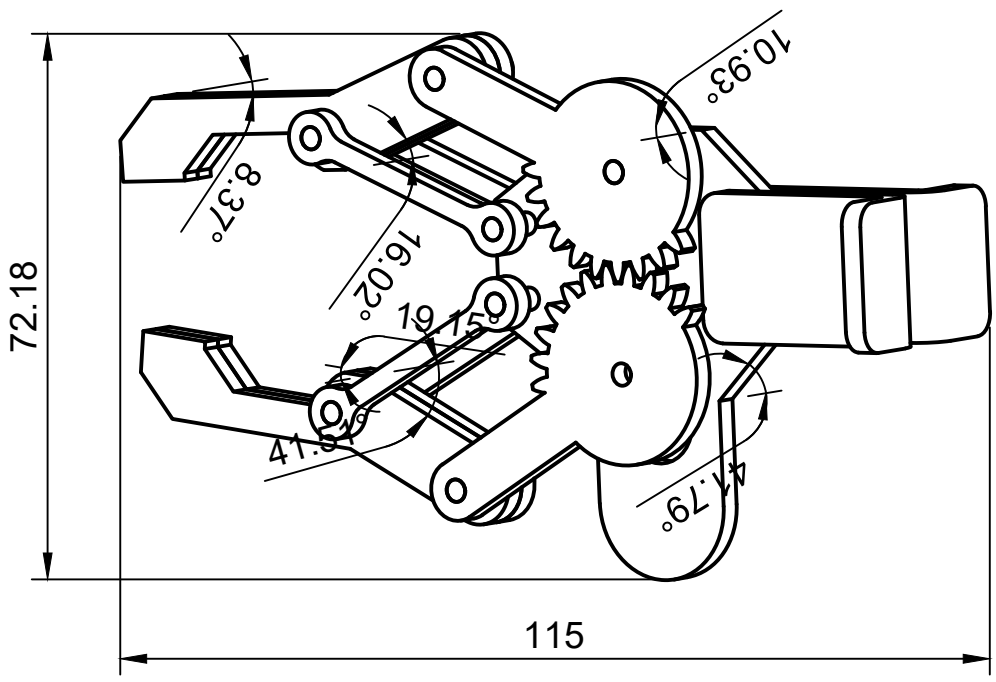
Dept.	Technical reference	Created by	Approved by
		G5 20-04-2026	
		Document type	Document status
		Title: Wrist Assemble	DWG No.: 13
		Rev.	Date of issue / Sheet: 1/1



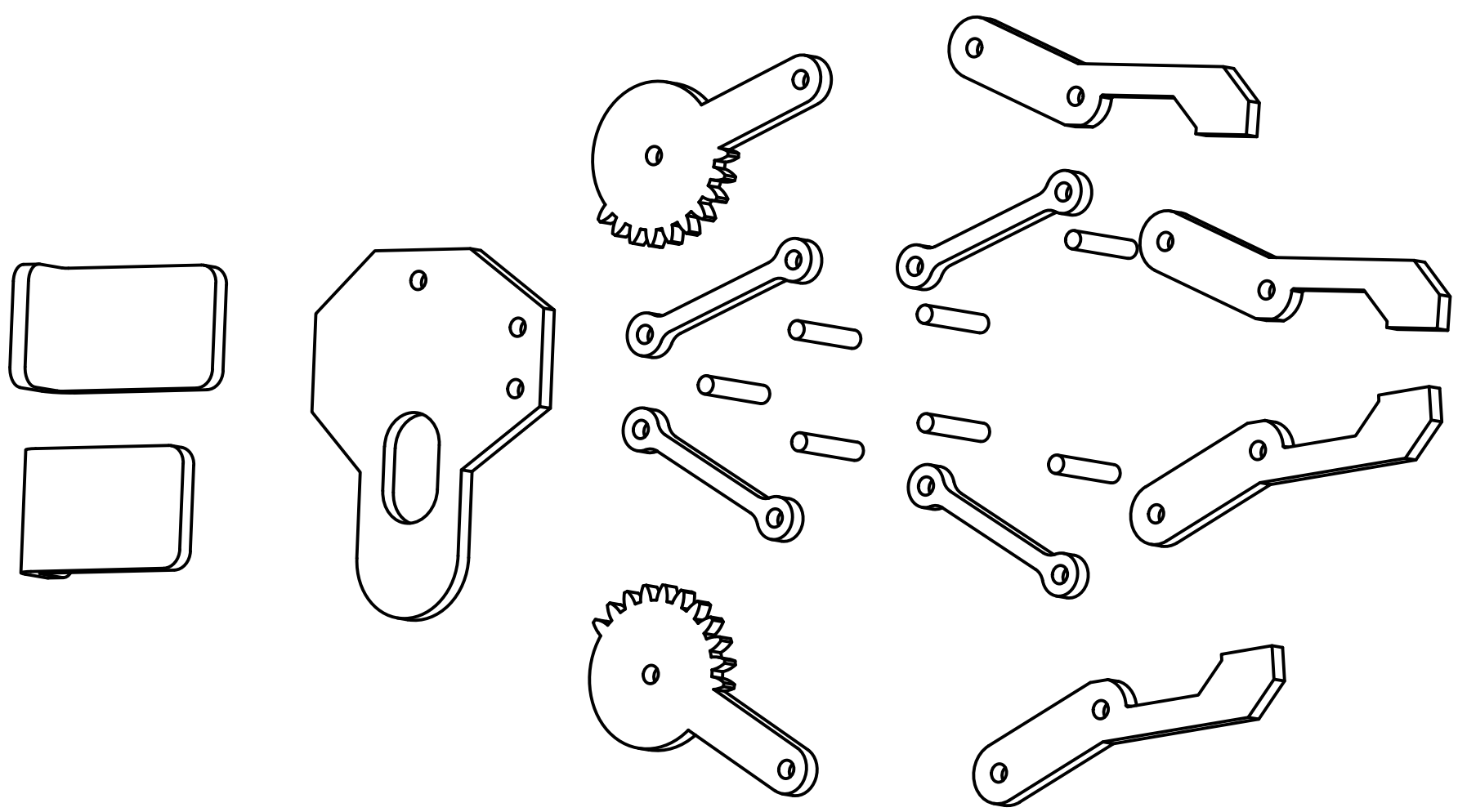


Dept.	Technical reference	Created by	Approved by
		G5 20-04-2026	
		Document type	Document status
		Title: Shaft	DWG No.: 15
			Rev. / Date of issue / Sheet: 1/1



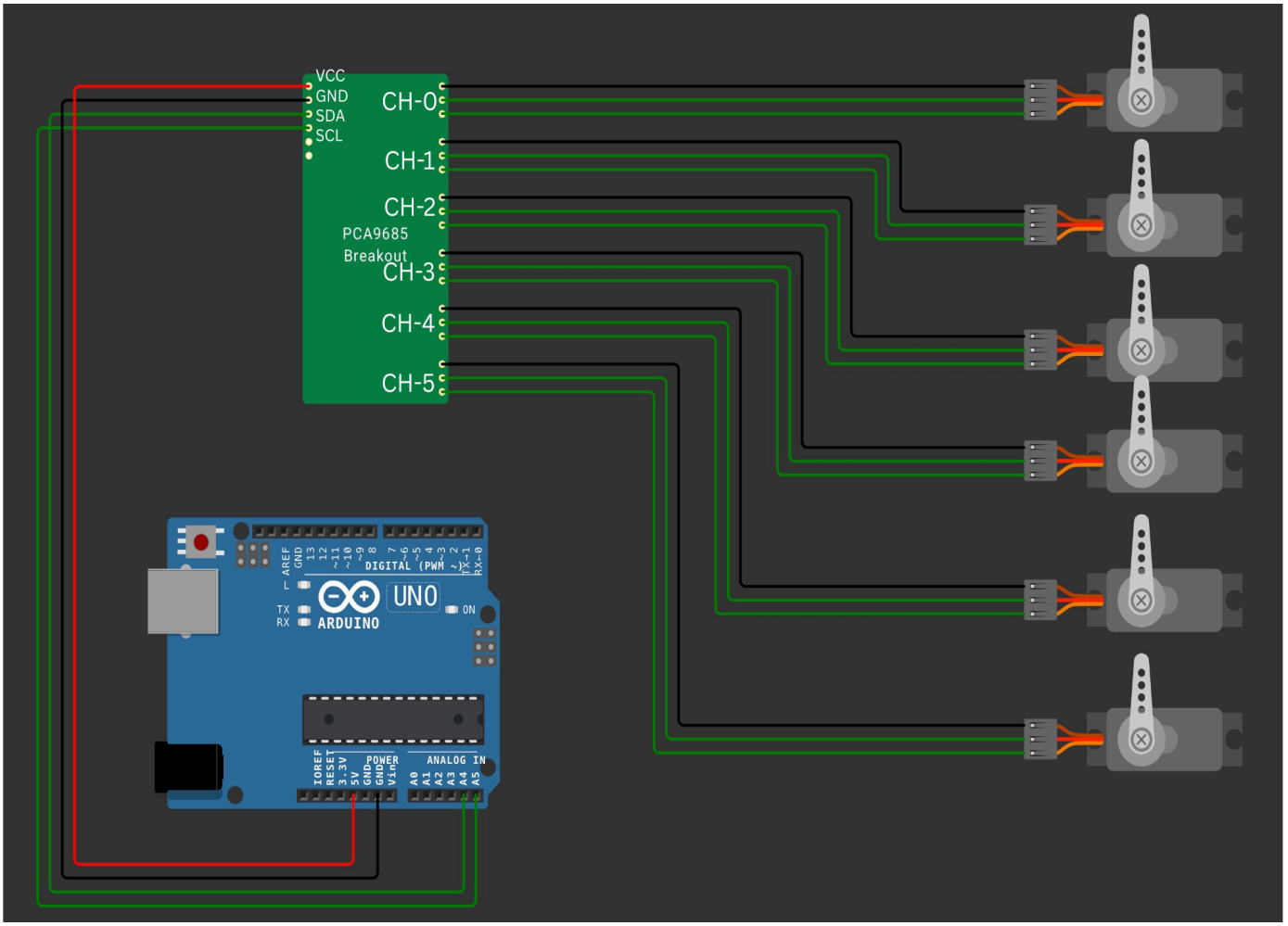


Dept.	Technical reference	Created by	Approved by
		G5 20-04-2026	
		Document type	Document status
		Title: Gripper	DWG No.: 16
			Rev. / Date of issue / Sheet: 1/1



Dept.	Technical reference	Created by	Approved by
		G5 20-04-2026	
		Document type	Document status
		Title: Gripper parts	DWG No.: 16
			Rev. / Date of issue / Sheet: 1/1

Circuit Diagram



Servo Motor Driver (PCA9685) to Arduino Uno Connections:

VCC → 5V
GND → GND
SDA → A4
SCL → A5

Servo Motor Driver (PCA9685) to Servo Motor Connections:

| CH0 → Base
| CH1 → Shoulder
| CH2 → Elbow
| CH3 → Wrist Pitch
| CH4 → Wrist Roll
| CH5 → Claw

INTRODUCTION-

This project controls a 6-DOF robotic arm using hand gestures captured through a webcam. The system has two main parts:

Python Code (CV)– Detects hand gestures and calculates servo angles.

```
import serial
import cv2
import mediapipe as mp
import time
import math

# ================= CONFIG =================
SERIAL_PORT = '/dev/cu.usbmodem101'  # CHANGE THIS
BAUD_RATE = 115200
debug = False
cam_source = 0

# ================= SERIAL =================
if not debug:
    ser = serial.Serial(SERIAL_PORT, BAUD_RATE, timeout=1)
    time.sleep(2)

# ================= LIMITS =================
x_min, x_max = 0, 180
y_min, y_max = 0, 180
z_min, z_max = 10, 180
wp_min, wp_max = 0, 180
wr_min, wr_max = 0, 180

claw_open = 60
claw_close = 0

fist_threshold = 7

# Initial
servo = [90, 90, 90, 90, 90, claw_open]
prev = servo.copy()

# ================= MEDIAPIPE =================
mp_hands = mp.solutions.hands
mp_draw = mp.solutions.drawing_utils
```

```python
cap = cv2.VideoCapture(cam_source)

# ================ HELPERS ================
def clamp(v, mn, mx):
    return max(min(v, mx), mn)

def map_range(x, in_min, in_max, out_min, out_max):
    return (x - in_min) * (out_max - out_min) / (in_max - in_min) + out_min

def dist(a, b):
    return ((a.x-b.x)**2 + (a.y-b.y)**2 + (a.z-b.z)**2)**0.5

# ================ FIST ================
def is_fist(hand, palm):
    WRIST = hand.landmark[0]
    s = 0
    for i in [8,12,16,20]:
        s += dist(WRIST, hand.landmark[i])
    return (s / palm) < fist_threshold

# ================ MAIN MAPPING ================
def get_servo(hand):
    angles = [90]*6

    WRIST = hand.landmark[0]
    INDEX = hand.landmark[5]
    MIDDLE = hand.landmark[9]

    palm = dist(WRIST, INDEX)

    # ===== CLAW =====
    angles[5] = claw_close if is_fist(hand, palm) else claw_open

    # ===== BASE =====
    dx = WRIST.x - INDEX.x
    base = clamp(dx * 300, -90, 90)
```

```
    angles[0] = int(map_range(base, -90, 90, x_max, x_min))

    # ===== SHOULDER =====
    y = clamp(WRIST.y, 0.3, 0.9)
    angles[1] = int(map_range(y, 0.3, 0.9, y_max, y_min))

    # ===== ELBOW =====
    palm = clamp(palm, 0.1, 0.3)
    angles[2] = int(map_range(palm, 0.1, 0.3, z_max, z_min))

    # ===== WRIST PITCH =====
    dy = WRIST.y - MIDDLE.y
    pitch = clamp(dy * 300, -90, 90)
    angles[3] = int(map_range(pitch, -90, 90, wp_min, wp_max))

    # ===== WRIST ROLL =====
    dx2 = hand.landmark[17].x - hand.landmark[5].x
    roll = clamp(dx2 * 300, -90, 90)
    angles[4] = int(map_range(roll, -90, 90, wr_min, wr_max))

    return angles

================ LOOP ================
ith mp_hands.Hands(min_detection_confidence=0.5,
                   min_tracking_confidence=0.5) as hands:

    while cap.isOpened():
        ret, frame = cap.read()
        if not ret:
            continue

        frame = cv2.flip(frame, 1)
        rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
        res = hands.process(rgb)
```

```
        if res.multi_hand_landmarks:
            hand = res.multi_hand_landmarks[0]
            new = get_servo(hand)

            # smoothing
            for i in range(6):
                servo[i] = int(0.7*prev[i] + 0.3*new[i])

            if servo != prev:
                print("Servo:", servo)

                if not debug:
                    data = ",".join(map(str, servo)) + "\n"
                    ser.write(data.encode())

                prev = servo.copy()

            mp_draw.draw_landmarks(frame, hand, mp_hands.HAND_CONNECTIONS)

        cv2.putText(frame, str(servo), (10,30),
                    cv2.FONT_HERSHEY_SIMPLEX, 1, (0,0,255), 2)

        cv2.imshow("6DOF Control", frame)

        if cv2.waitKey(5) & 0xFF == 27:
            break

cap.release()
cv2.destroyAllWindows()
```

Arduino Code – Receives the servo angles and controls the servo motors using a PCA9685 driver.

```ino
#include <Wire.h>
#include <Adafruit_PWMServoDriver.h>

Adafruit_PWMServoDriver pca = Adafruit_PWMServoDriver();

// ===== SERVO LIMITS =====
#define SERVOMIN  150   // pulse for 0°
#define SERVOMAX  600   // pulse for 180°

int angles[6] = {90, 90, 90, 90, 90, 60};   // current
int target[6] = {90, 90, 90, 90, 90, 60};   // incoming

String input = "";

// ===== MAP ANGLE → PWM =====
int angleToPulse(int angle) {
  return map(angle, 0, 180, SERVOMIN, SERVOMAX);
}

void setup() {
  Serial.begin(115200);

  pca.begin();
  pca.setPWMFreq(50);  // standard servo frequency

  delay(500);

  // Initialize servos to default position
  for (int i = 0; i < 6; i++) {
    pca.setPWM(i, 0, angleToPulse(angles[i]));
  }
}
```

```
void loop() {

  // ===== READ SERIAL =====
  if (Serial.available()) {
    input = Serial.readStringUntil('\n');

    int i = 0;
    char *token = strtok((char*)input.c_str(), ",");

    while (token != NULL && i < 6) {
      target[i++] = atoi(token);
      token = strtok(NULL, ",");
    }
  }

  // ===== SMOOTH MOVEMENT =====
  for (int i = 0; i < 6; i++) {
    angles[i] = 0.7 * angles[i] + 0.3 * target[i];
  }

  // ===== APPLY TO PCA9685 =====
  for (int ch = 0; ch < 6; ch++) {
    int pulse = angleToPulse(angles[ch]);
    pca.setPWM(ch, 0, pulse);
  }

  delay(20);  // smooth update rate
}
```

The webcam tracks the hand in real time. Python processes the hand movement and sends corresponding servo angles to the Arduino. The Arduino then moves the robotic arm accordingly.

Python Code Explanation

1. Importing Libraries
The Python code uses the following libraries:

- OpenCV (cv2) – For capturing webcam video and displaying it.
- MediaPipe – For detecting 21 hand landmarks.
- Serial – For sending data to Arduino through USB.
- Time and Math – For delay and calculations.

2. Configuration Section
In the beginning, some configuration values are defined:

- Serial port and baud rate (115200).
- Debug mode (to enable or disable serial communication).
- Camera source.

This section sets up how the program connects to the Arduino and which camera it uses.

3. Serial Communication Setup
If debug mode is set to False, the program opens the serial port:

The serial connection allows Python to send servo angles to the Arduino continuously.

4. Servo Limits and Initial Position
Servo limits are defined between 0 and 180 degrees to prevent damage.

Initial servo positions are set as:

- Base: 90°
- Shoulder: 90°
- Elbow: 90°
- Wrist Pitch: 90°
- Wrist Roll: 90°
- Claw: 60° (open)

This ensures the robotic arm starts from a safe neutral position.

5. Helper Functions
There are three important helper functions:

- Clamp function:
  This ensures values stay within a safe range.

- Map function:
  This converts one range of values into another range. For example, it converts hand movement values into servo angle values.

- Distance function:
  This calculates the 3D distance between two hand landmarks.

6. Fist Detection
The function checks whether the hand is closed.

It calculates the distance between the wrist and fingertips (index, middle, ring, and pinky).

If the fingers are close to the wrist, the program detects a fist and closes the claw servo.
If the hand is open, the claw remains open.

7. Mapping Hand Movements to Servo Angles
The function get_servo() converts hand landmark positions into six servo angles.

Each servo is controlled as follows:

- Servo 0 – Base
  Controlled by left and right movement of the hand.

- Servo 1 – Shoulder

Controlled by up and down movement of the wrist.

- Servo 2 – Elbow
  Controlled by palm size (distance between wrist and index finger).
  If the hand moves closer to the camera, the elbow angle changes.

- Servo 3 – Wrist Pitch
  Controlled by bending the hand up or down.

- Servo 4 – Wrist Roll
  Controlled by rotating the hand.

- Servo 5 – Claw
  Controlled by fist detection (open or close).

8. Smoothing the Movement
To avoid jerky movement, smoothing is applied:

New Angle = 0.7 × Previous Angle + 0.3 × Current Angle

This makes the robotic arm move smoothly instead of suddenly jumping to new positions.

9. Sending Data to Arduino
The servo angles are converted into a comma-separated string like:

90,120,45,80,90,60

This string is sent to the Arduino using serial communication at 115200 baud rate.

Arduino Code Explanation

1. PCA9685 Servo Driver
The Arduino uses a PCA9685 PWM driver to control 6 servo motors.
The PWM frequency is set to 50 Hz, which is standard for servo motors.

2. Reading Serial Data
Arduino continuously checks if data is available on the serial port.

When data is received, it reads one full line and separates the six angle values using commas. These values are stored as target angles.

3. Smooth Servo Movement
Arduino again applies smoothing:

Angle = 0.7 × Current Angle + 0.3 × Target Angle

This ensures smooth and stable movement of the robotic arm.

4. Converting Angle to PWM
Servo motors do not directly understand angles.
So, angles (0–180 degrees) are converted into PWM pulse values:

150 corresponds to 0 degrees
600 corresponds to 180 degrees

These pulse values are sent to the PCA9685 driver to move the servos.

5. Continuous Loop
The Arduino loop runs continuously with a small delay (20 milliseconds). This keeps updating servo positions in real time according to hand movement.

Overall Working:

1. Webcam captures the hand.
2. MediaPipe detects 21 landmarks.
3. Python calculates servo angles based on hand position.
4. Angles are smoothed.
5. Angles are sent to Arduino via serial communication.
6. Arduino reads the angles.
7. PCA9685 drives the servo motors.
8. The robotic arm moves according to the hand gesture.

Bill Of Materials and Cost Analysis

S. No.	Component	Quantity	Availability	Cost in Lab	Actual Cost
1	Servo Motor (SG90)	3	Available	0	150*3=450
2	Servo Motor (MG996R)	3	Purchased	650*3=1950	650*3=1950
3	Servo Motor Driver (PCA9685)	1	Purchased	300*1=300	300*1=300
4	Arduino Uno	1	Available	0	500*1=500
5	Battery Holder	1	Available	0	30*1=30
6	Li-ion Batteries (3.7V)	2	Available	0	50*2=100
7	Jumper Wires	Multiple	Available	0	~100

Cost of Electrical Components purchased = 1950+300 = **2250**
Actual total cost of Electrical Components = **3430**

S. No.	Part No.	Part Name	Quantity	Process\ Material Used	Time Taken (Hrs)	Cost
1	1	Mounting 1	1	3D Printing (PLA)	1.5	100*1.5=150
2	3	Base Support	3	3D Printing (PLA)	1.5	100*1.5=150
3	4	Gear (a,b)	2	3D Printing (PLA)	1	100*1=100
4	5	Base Plate A	1	Turning (ACP)	2	150*2=200
5	6	Spacer Plate	3	3D Printing (PLA)	2	100*2=200
6	7	Base Plate B	2	3D Printing (PLA)	1.5	100*1.5=150
7	8	Mounting 2	1	3D Printing (PLA)	1.5	100*1.5=150
8	9	Arm (a,b)	2	3D Printing (PLA)	2.5	100*2.5=250
9	11	Front Arm (a,b)	2	3D Printing (PLA)	2	100*2=200
10	12	Upper Arm Spacer	2	3D Printing (PLA)	1.25	100*1.25=125
11	15	Shaft	1	Turning (Metal)	1.5	150*1.5=225
12	16	Gripper	1	3D Printing (PLA)	3	100*3=300

Material cost (750g PLA +100g Metal +400g ACP)= 500*0.75+100*0.1+350*0.4=**525**
Total cost of Machining = **2200**
Total Cost of Manufactured Components (Material+Machining) = 2200+525 = **2725**

Actual Value of the Project (Electrical + Mechanical) = 2725+3430 = *Rs 6155*